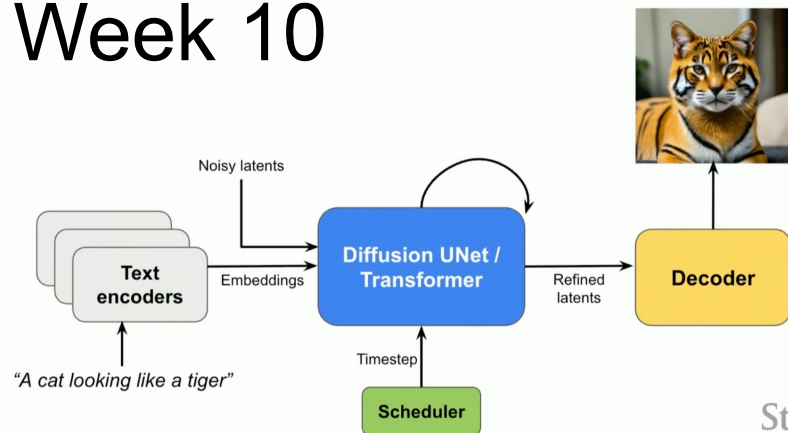


Transformers in Deep Learning

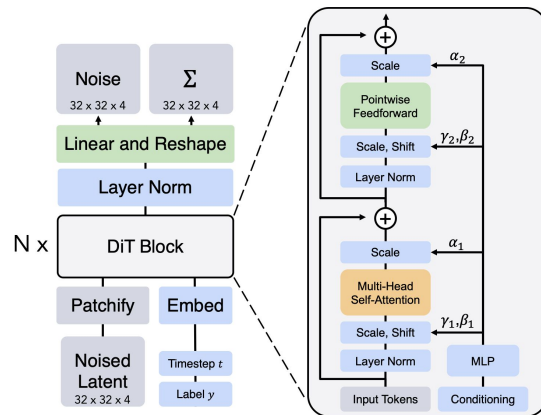
Lecture 10

Week 10



Stanford

Diffusion Transformer



<https://arxiv.org/abs/2212.09748>

Forward Process

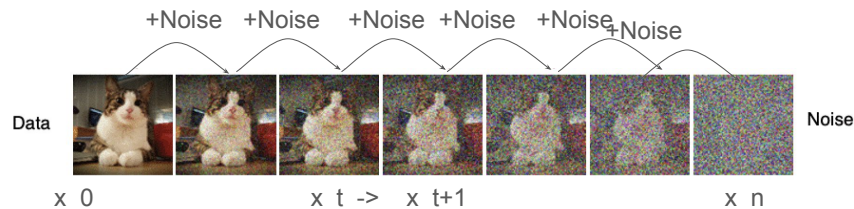
During training, noise is gradually added to the clean image. At timestep t , the noisy image x_t is constructed as

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon,$$

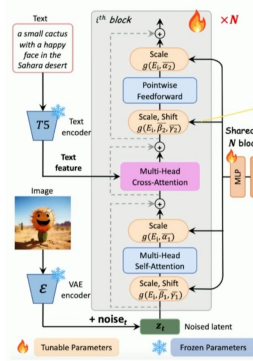
where

$$\epsilon \sim \mathcal{N}(0, I),$$

and $\bar{\alpha}_t$ controls how much signal remains from the original image.



Enabling T2I in DiT (DiT is class conditioned)



- How to embed the input prompts?
 - Use a text encoder.
- How to learn the contextual dependencies? We have text + timestep.
 - Self-attention on the noisy latents.
 - Use cross-attention between noisy latents and text.
 - Modulate just the timestep.
- Initialize from a class-conditional counterpart if available and compatible.

[PixArt-α: Fast Training of Diffusion Transformer for Photorealistic Text-to-Image Synthesis](#)

Stanford

Era of Text-to-Image Diffusion Models



"A small cactus with a happy face in the Sahara desert."

PixArt-Alpha



"panda mad scientist mixing sparkling chemicals, digital art."

DALL-E 3



"A tiny astronaut hatching from an egg on the moon"

Flux.1 Merged

Diffusion models



Denoising Process with Unimodal Gaussian Distribution

Data

Noise

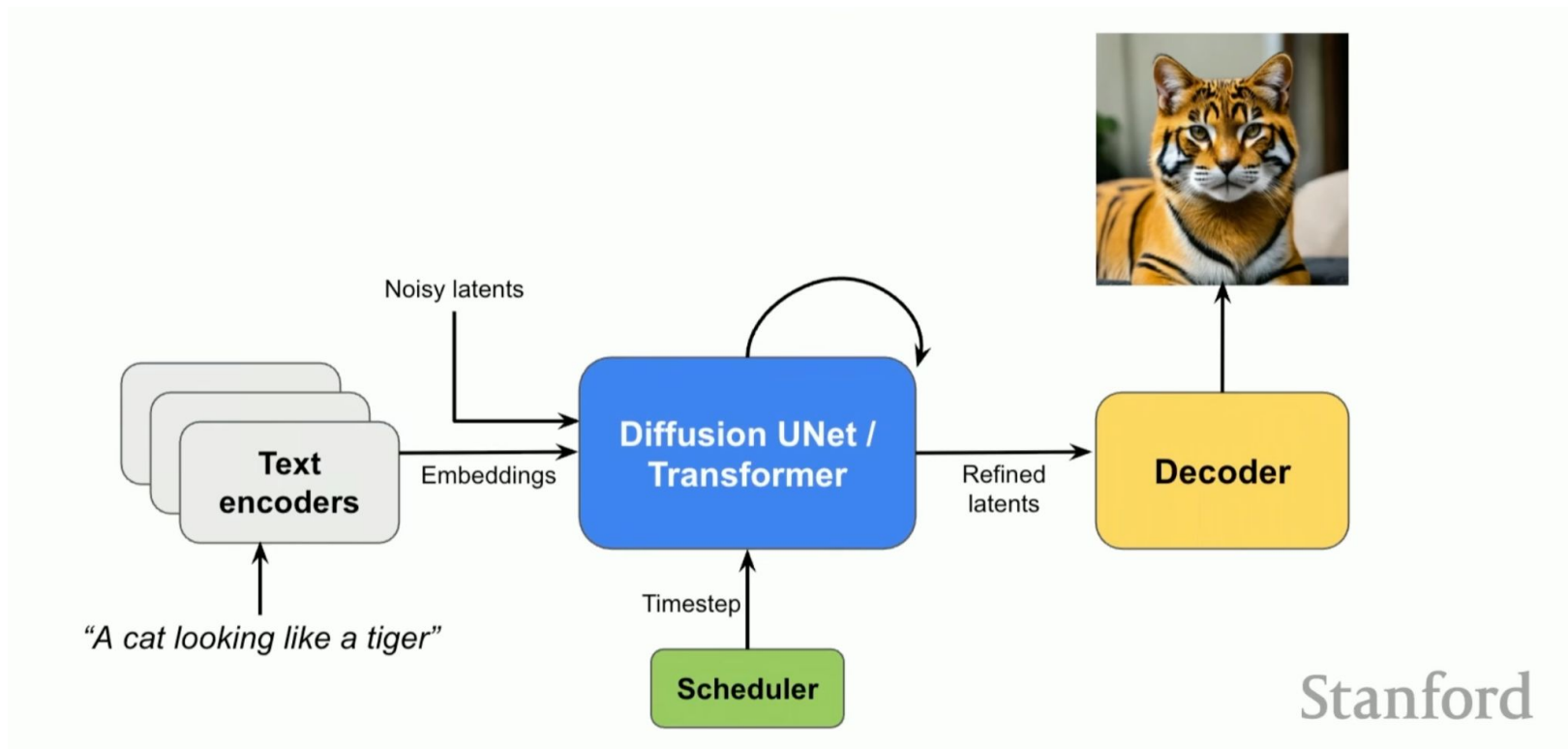
Diffusion models

You can “condition” diffusion models with text



Flux prompt: “A photo of a white fur monster standing in a purple room”

Diffusion models are NOT single models

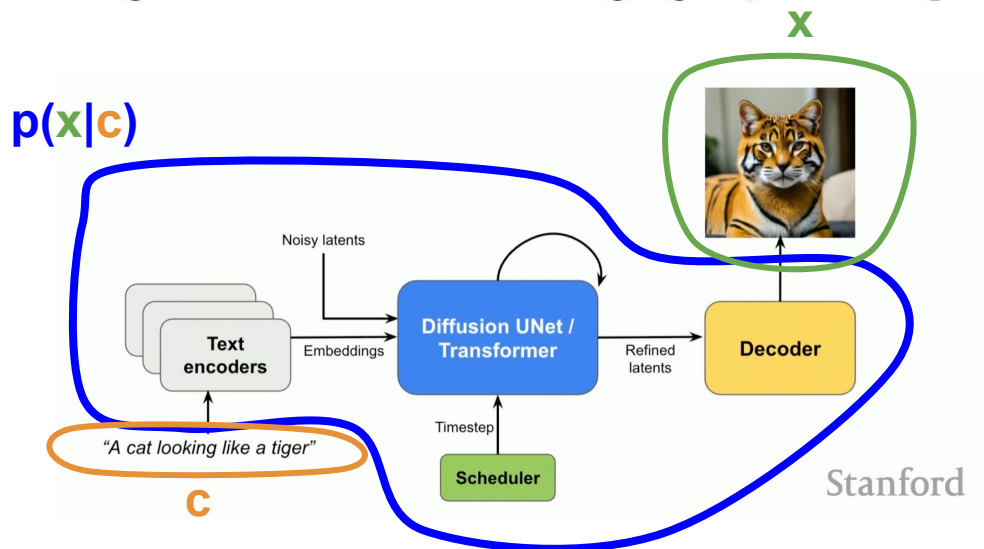


Diffusion models

We model the conditional distribution of an image

$$p(x_0 \mid c),$$

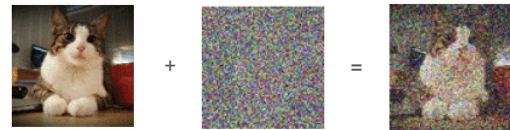
where x_0 is the clean image and c is the conditioning signal, for example a text prompt, class label, or another image.



Forward Process

During training, noise is gradually added to the clean image. At timestep t , the noisy image x_t is constructed as

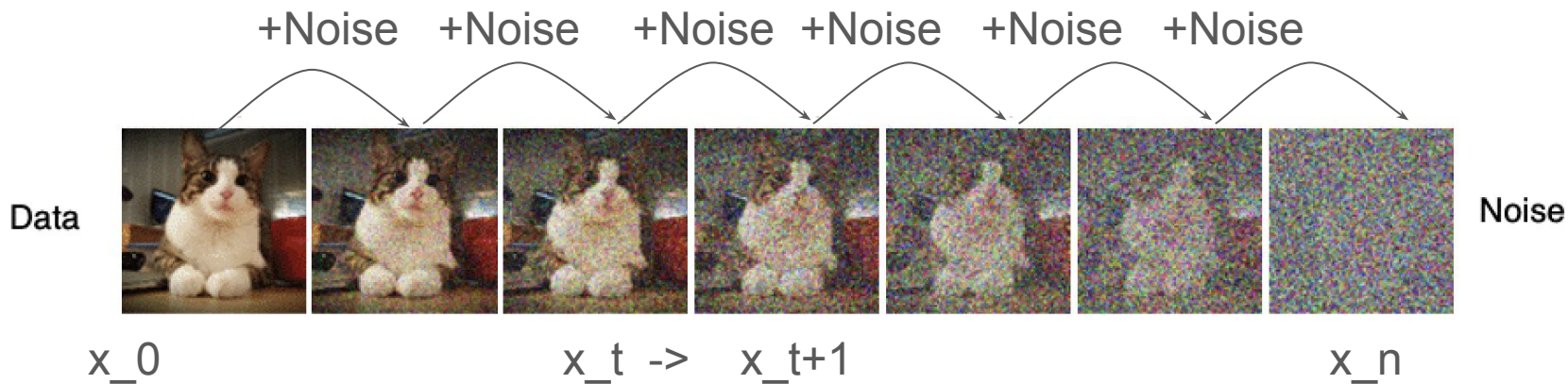
$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon,$$



where

$$\epsilon \sim \mathcal{N}(0, I),$$

and $\bar{\alpha}_t$ controls how much signal remains from the original image.



Objective

A neural network $\epsilon_\theta(x_t, t, c)$ is trained to predict the noise that was added to the image:

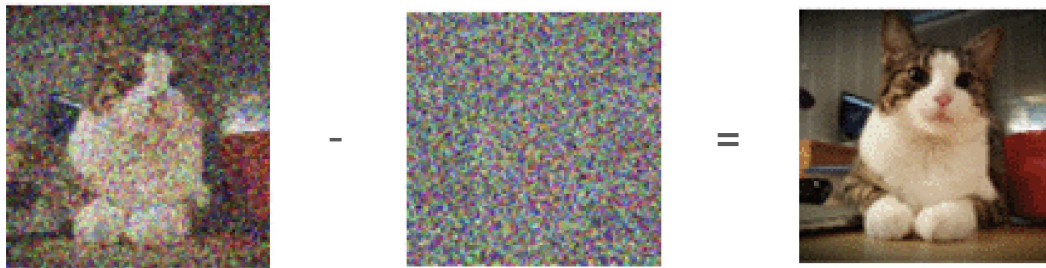
$$L(\theta) = \mathbb{E}_{x_0, \epsilon, t} [\|\epsilon - \epsilon_\theta(x_t, t, c)\|^2] .$$

Here:

- x_t is the noisy image,
- t is the diffusion timestep,
- c is the conditioning information,
- $\epsilon_\theta(x_t, t, c)$ is the predicted noise.

So the model learns how to remove noise from partially corrupted images.

Objective



$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t, c) \right) + \sigma_t z,$$

Generation

At inference time, generation starts from pure Gaussian noise:

$$x_T \sim \mathcal{N}(0, I).$$

The model then iteratively denoises the sample:

$$x_{t-1} = \text{Denoise}(x_t, \epsilon_\theta(x_t, t, c)),$$

moving step by step from noise toward a clean image.

In a simplified view, each step uses the model prediction to estimate what part of x_t is noise and what part is signal. After many denoising steps, the final sample x_0 becomes a coherent image matching the condition c .

Data

Noise

Intuition

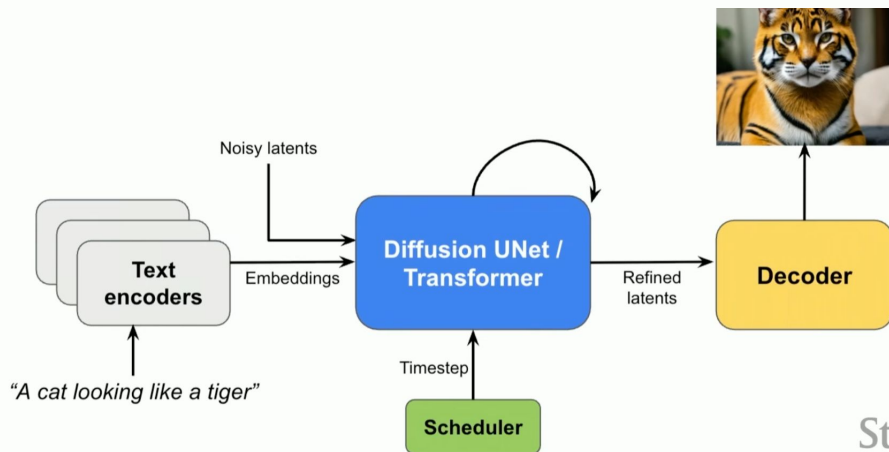
- During training, noise is added to real images.
- The model learns to predict the noise that was added.
- During inference, generation starts from random noise.
- The model removes noise step by step until a clean image appears.

Data

Noise

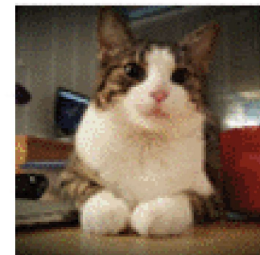
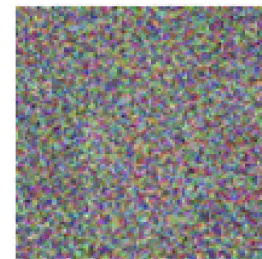
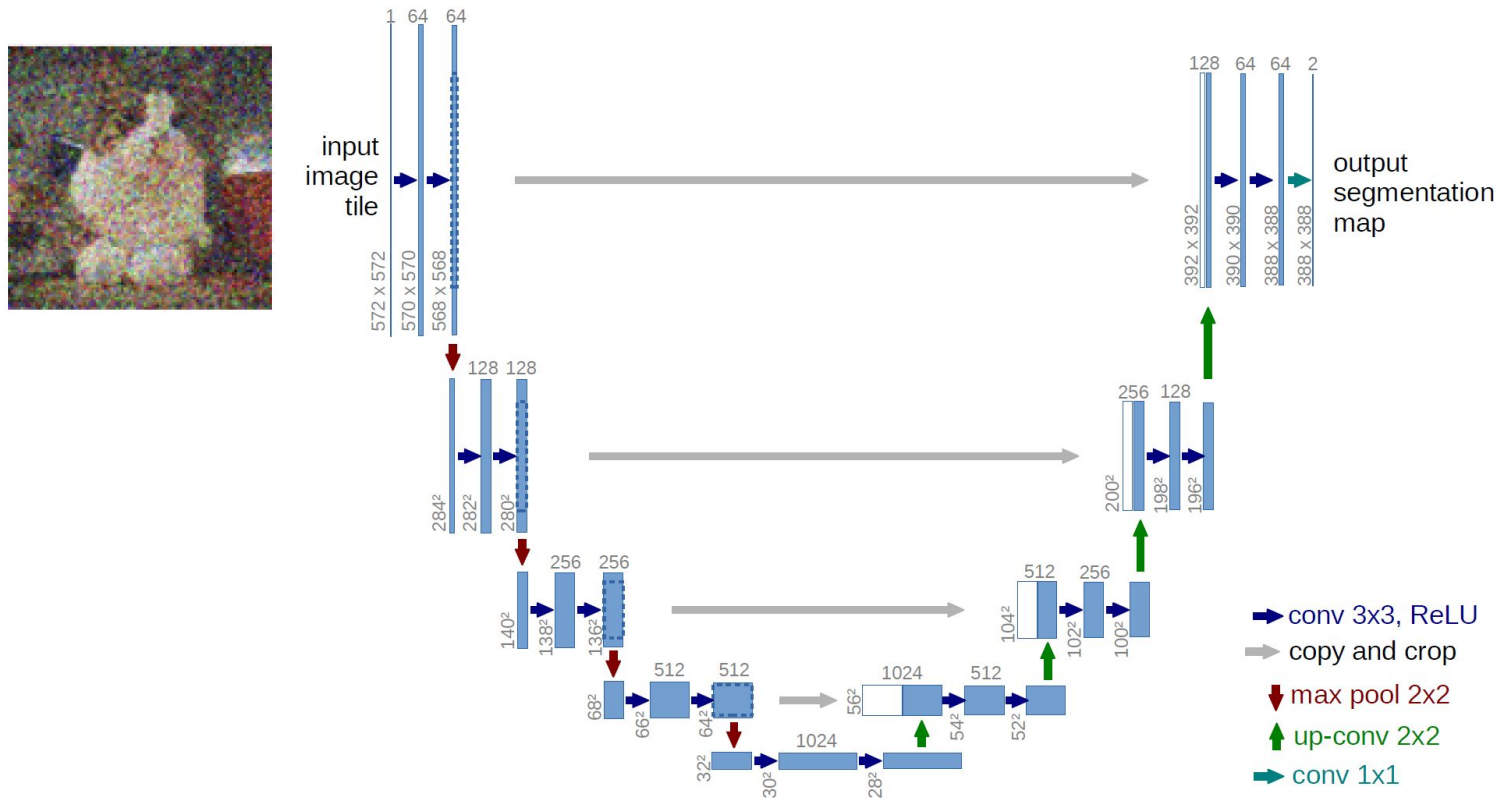
A diffusion model

- The early models (DDPM and LDM) used a UNet-based architecture.
- Historically speaking, UNets have always dominated this kind of space for images.
- So, a natural choice for the diffusion network has been UNet-based till SDXL.

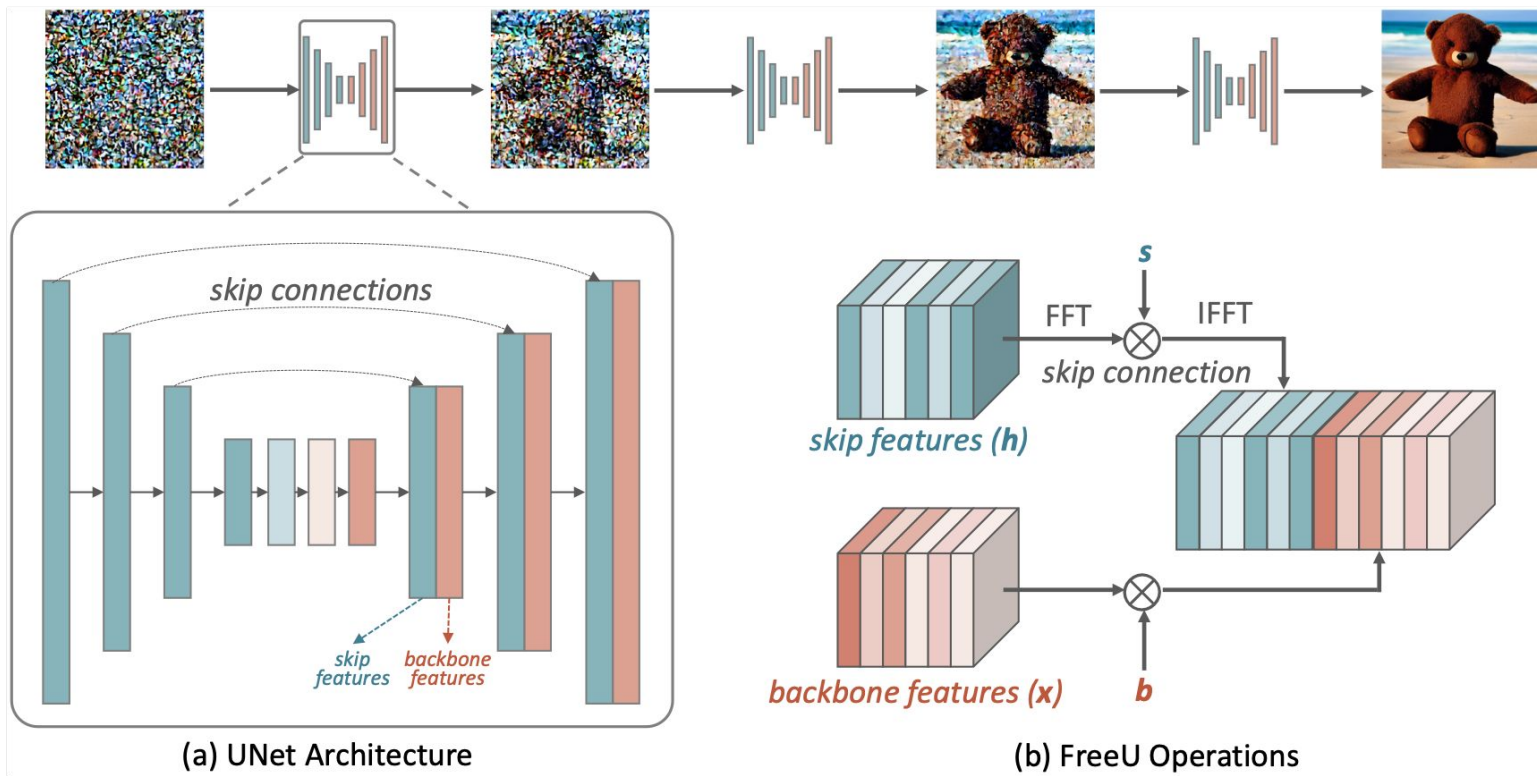


Stanford

U-net



U-net diffusion model



U-net

- An input convolution stem
- A series of downblocks
 - Custom residual blocks (comprising normalization, convolutions, etc.)
 - Custom transformer blocks (comprising normalization, projections, and regular transformers)
 - Convolution layers for upsampling
- A single midblock (similar but no resolution changes)
- A series of upblocks
 - Similar as downblocks with convolution layers for downsampling to match the input resolution
- Misc:
 - Timestep embeddings*
 - Additional embeddings (class embeddings, micro-conditions, text embeddings)

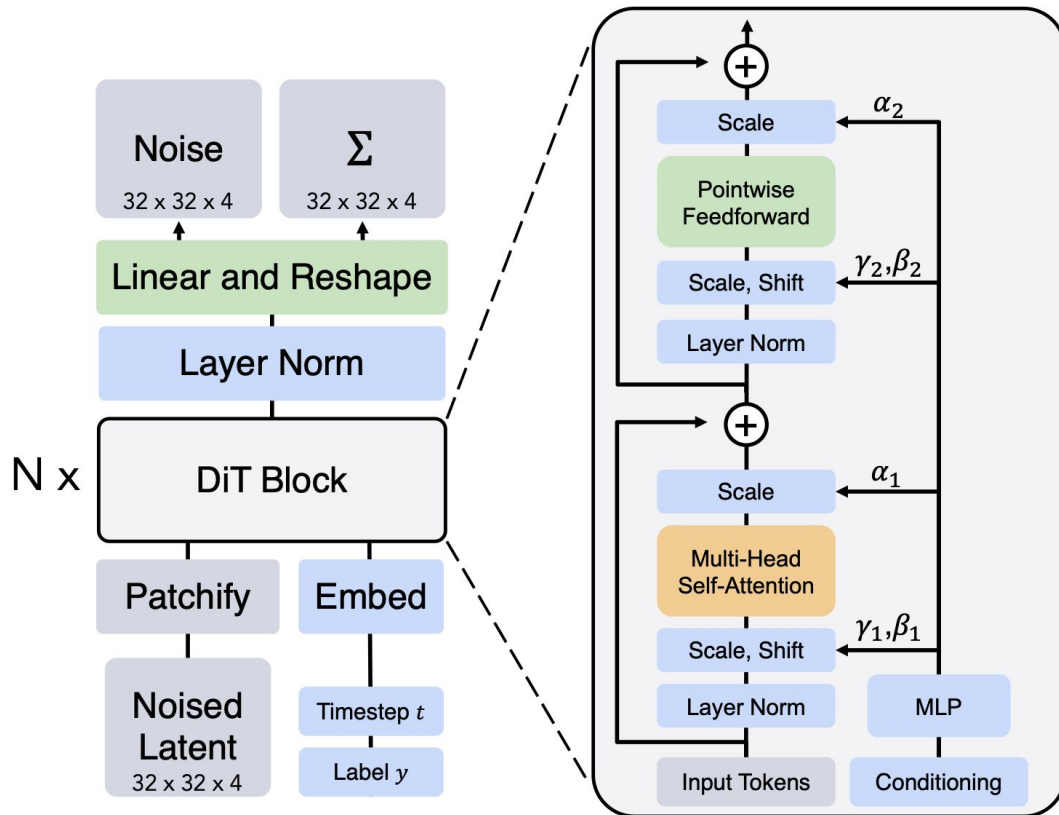
Stanford

*Is Noise Conditioning Necessary for Denoising Generative Models?

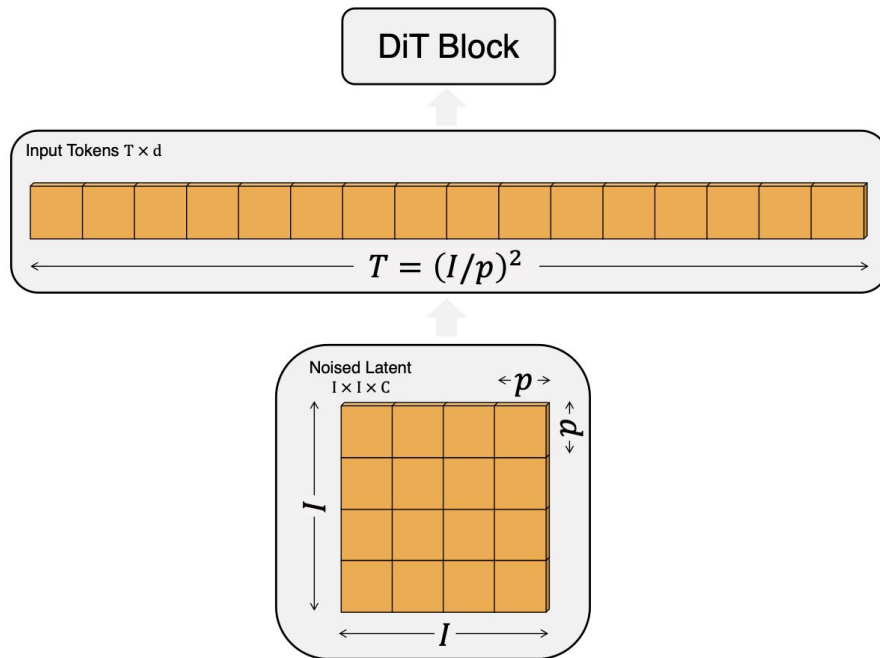
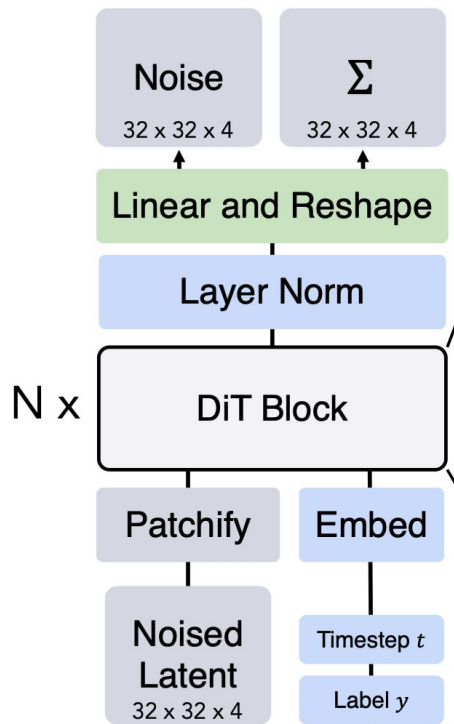
Why Transformer?

- Advancements in transformers benefit DiTs too (parallel-MLP, QK-Norm, GQA, fancy positional encoding, etc.).
- Known for good scaling properties.
- Allows for clean integration with other backbones (more on this later).
- Gets rid of the giant UNet.

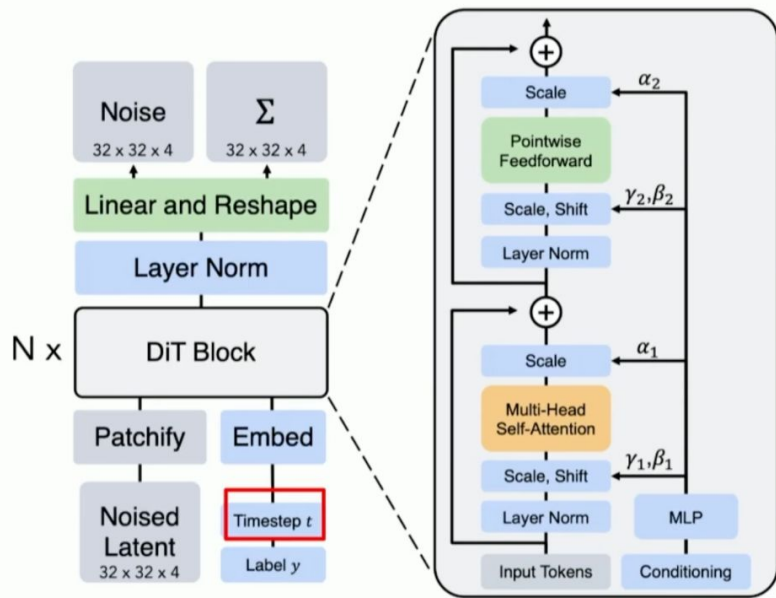
Diffusion Transformer



Diffusion Transformer



Diffusion Transformer a closer look



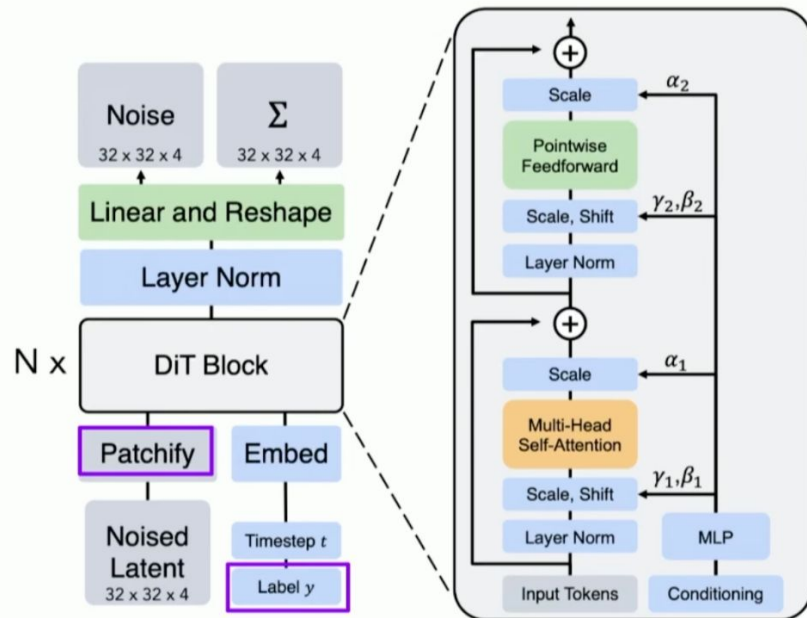
Embedding **timesteps**:

- Timestep ranges from $[0, 1000]$, denoting the noise-level.
- Each t is embedded into sinusoidal frequency space (phase-awareness).
- Then passes through a 2-layer non-linear (SiLU) MLP.

(B, D)

Stanford

Diffusion Transformer a closer look



Embedding **class-labels**:

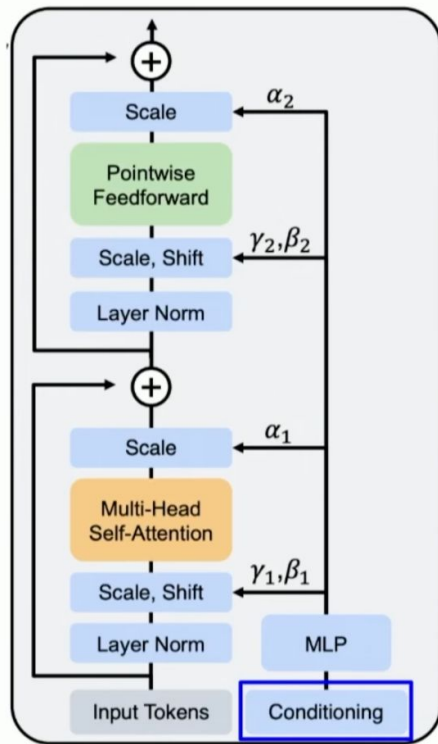
- Standard `nn.Embedding` layer with a dimensionality of DiT hidden size.
- (B, D)

Standard **patchification** + sin-cos pos embeddings (B, N, D)

$$N = H * W / \text{patch_size} ** 2$$

Vanilla DiT is class conditioned!!!

Diffusion Transformer a closer look



The **conditioning**:

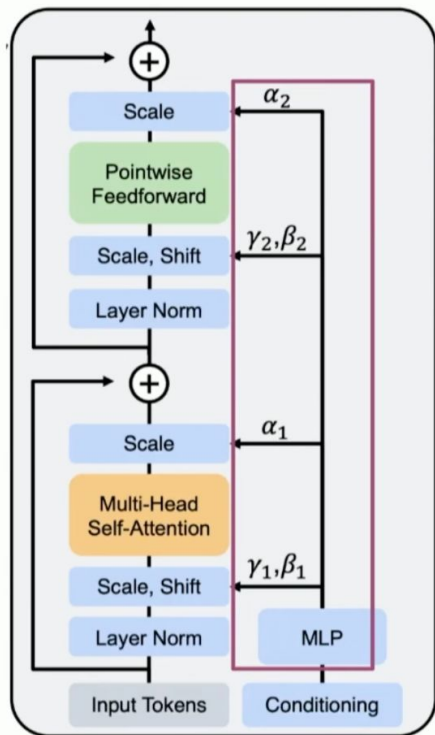
$$\begin{aligned} \mathbf{t}_{\text{emb}} &= \mathbf{t}_{\text{embed}}(t) \\ \mathbf{y}_{\text{emb}} &= \mathbf{y}_{\text{embed}}(y) \\ \mathbf{c} &= \mathbf{t}_{\text{emb}} + \mathbf{y}_{\text{emb}} \end{aligned}$$

$$(\mathbf{B}, \mathbf{D})$$

$\mathbf{c}$ remain **fixed** across blocks.

Stanford

Diffusion Transformer a closer look

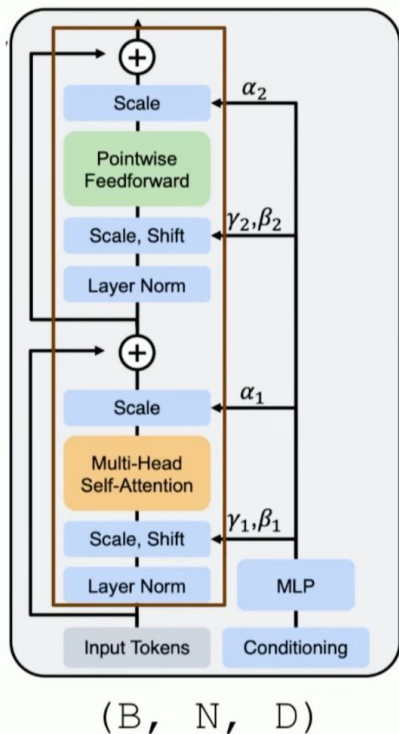


For each DiT block, **adaptive layer-norm** is regressed from the combined condition ($c = y + t$).

```
1  adaLN = nn.Sequential(  
2      nn.SiLU(),  
3      nn.Linear(hidden_size, 6 * hidden_size, bias=True)  
4  )  
5  gamma_msa, beta_msa, alpha_msa, gamma_mlp, beta_mlp, alpha_mlp = (  
6      adaLN(c).chunk(6, dim=1)  
7  )
```

Stanford

Diffusion Transformer a closer look



Skipping the usual transformer stuff (QKV, MSA, MLP), the governing equations:

$$\mathbf{x} \leftarrow \mathbf{x} + \text{Attn}(\text{LayerNorm}_1(\mathbf{x}))$$

$$\mathbf{x} \leftarrow \mathbf{x} + \text{MLP}(\text{LayerNorm}_2(\mathbf{x}))$$

But for a **DiT**,

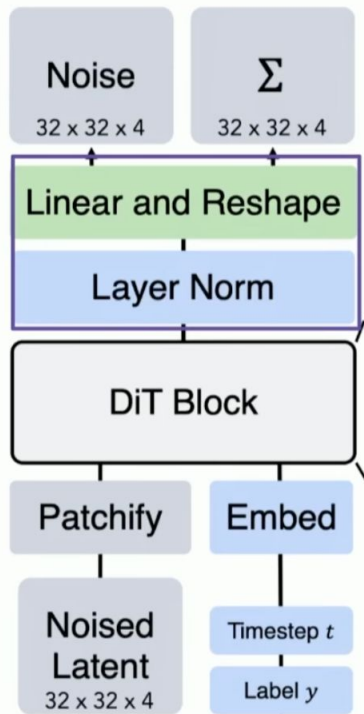
$$\mathbf{x} \leftarrow \mathbf{x} + \alpha_{\text{msa}} \text{Attn}(\text{modLN}_1(\mathbf{x}, \beta_{\text{msa}}, \gamma_{\text{msa}}))$$

$$\mathbf{x} \leftarrow \mathbf{x} + \alpha_{\text{mlp}} \text{MLP}(\text{modLN}_2(\mathbf{x}, \beta_{\text{mlp}}, \gamma_{\text{mlp}}))$$

$$\text{modLN}_i(\mathbf{x}, \beta, \gamma) = (\text{LayerNorm}_i(\mathbf{x})) \odot (1 + \gamma) + \beta$$

Stanford

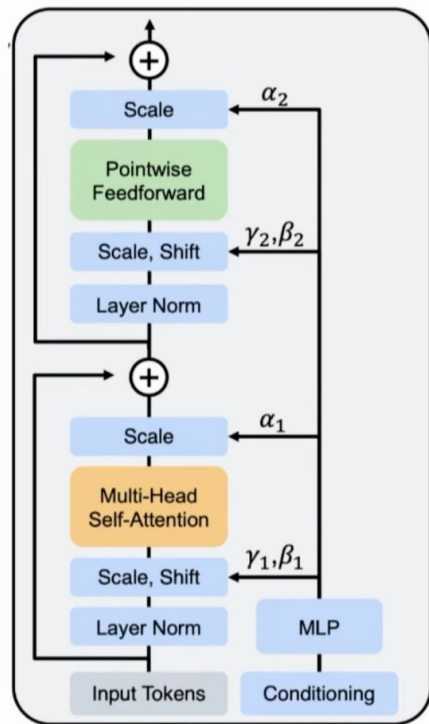
Diffusion Transformer a closer look



- **Single layer** to decode (B, N, D) into $(B, N, (\text{patch_size} ** 2) * \text{out_channels})$
- **Reshape** into $(B, \text{out_channels}, H, W)$

Stanford

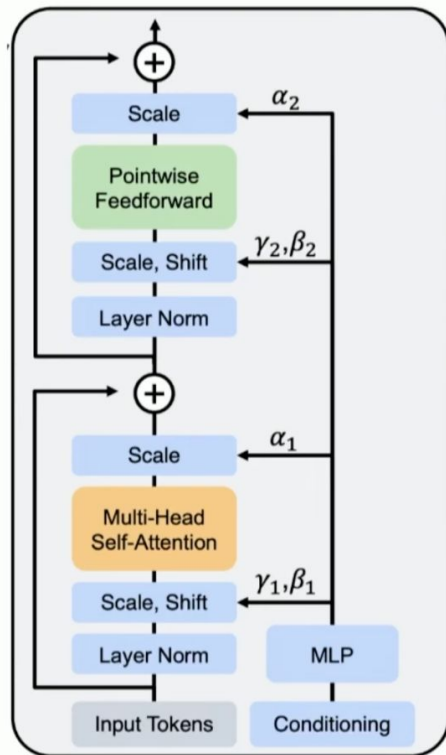
Diffusion Transformer a closer look



- Standard ViT init but ...
- Each DiT block is initialized as an identity block (for adaLN-zero).
- Final layer is also initialized as zero.

Stanford

Diffusion Transformer a closer look



- adaLN-zero is important and is compute-efficient.
- Later works have also explored sharing the adaLN parameters across multiple blocks, reducing compute.
- Conditioning modulation is paramount.

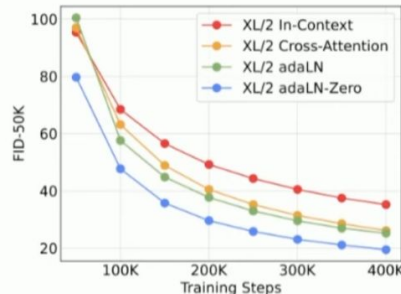
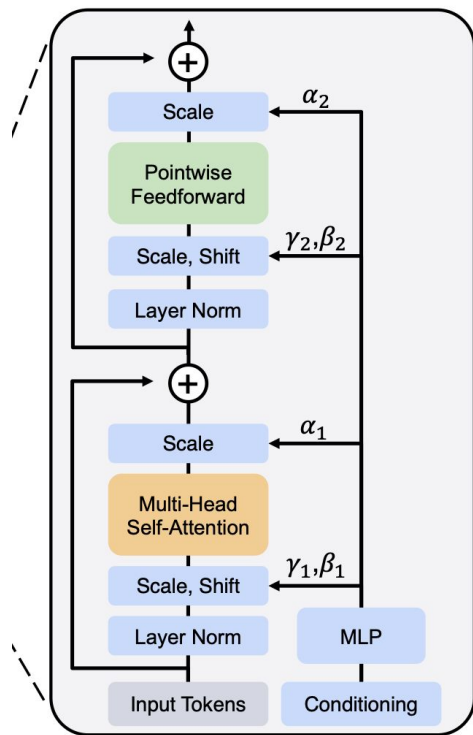


Figure 5. Comparing different conditioning strategies. adaLN-Zero outperforms cross-attention and in-context conditioning at all stages of training.

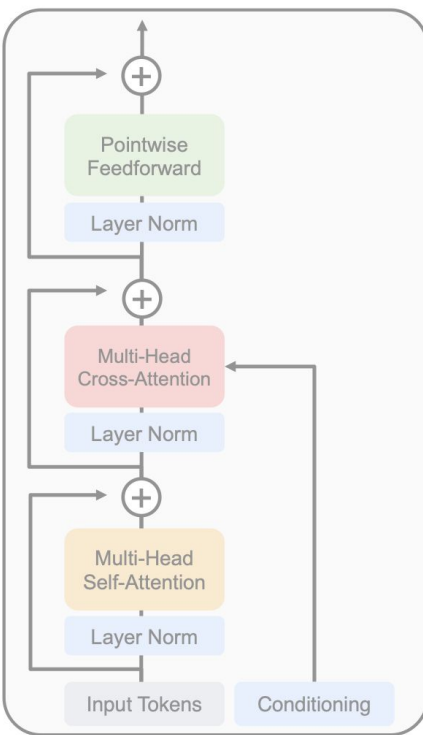
💡 Cross-attention didn't help.

Stanford

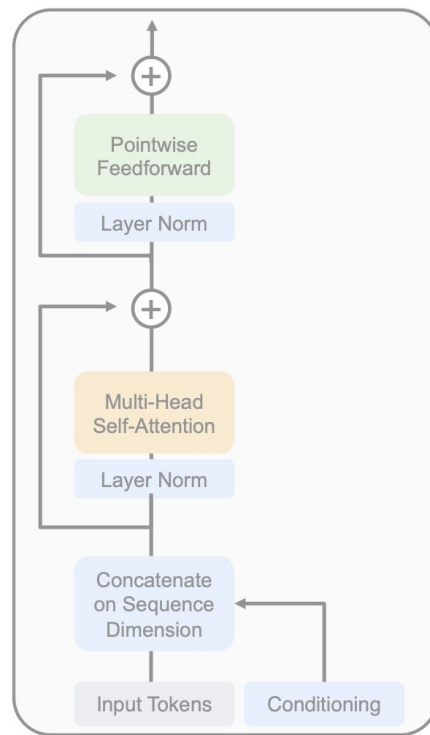
Diffusion Transformer a closer look



DiT Block with adaLN-Zero

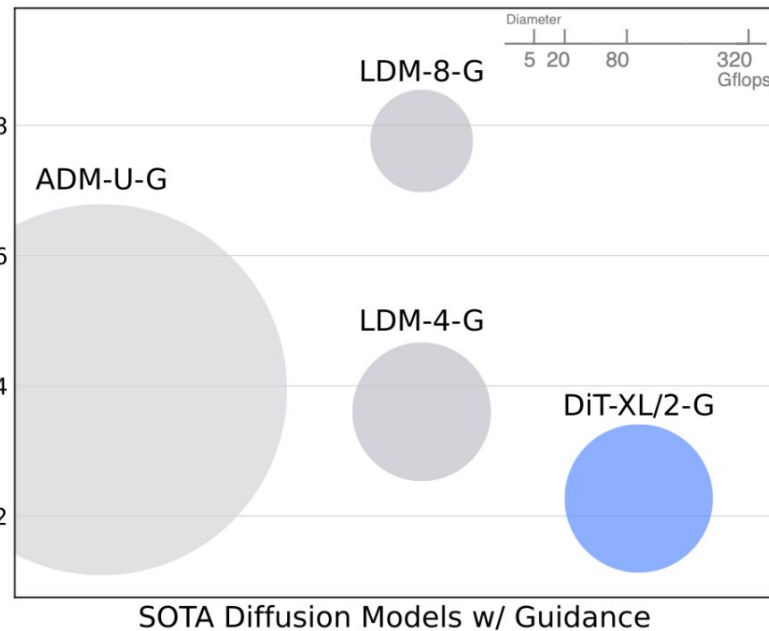
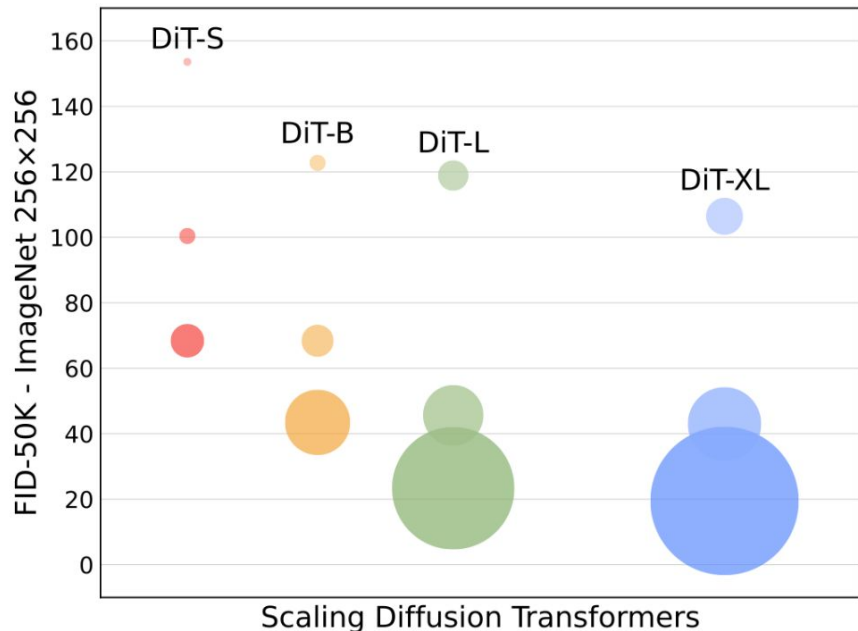


DiT Block with Cross-Attention

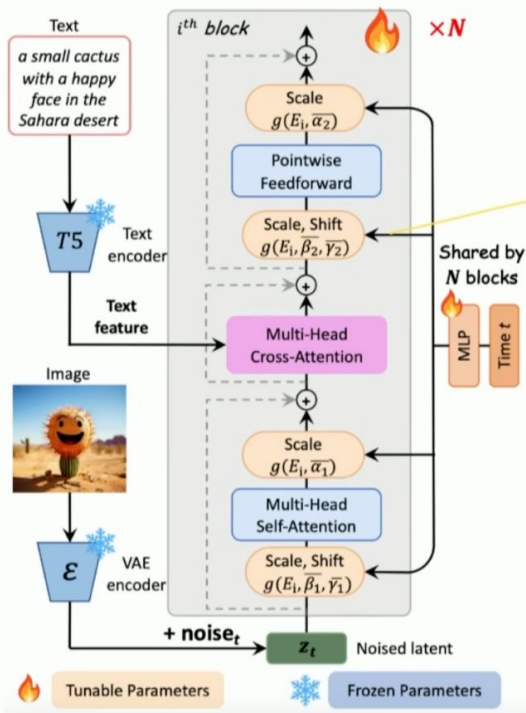


DiT Block with In-Context Conditioning

Diffusion Transformer scaling and efficiency



Enabling T2I in DiT (DiT is class conditioned)

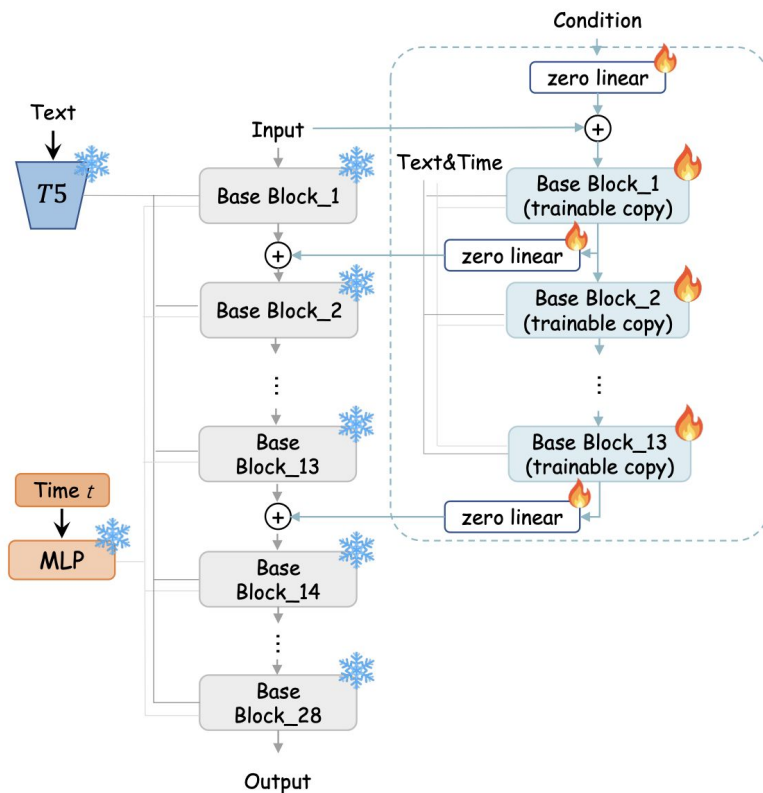


- How to embed the input prompts?
 - Use a text encoder.
- How to learn the contextual dependencies? We have text + timestep.
 - Self-attention on the noisy latents.
 - Use cross-attention between noisy latents and text.
 - Modulate just the timestep.
- Initialize from a class-conditional counterpart if available and compatible.

PixArt-α: Fast Training of Diffusion Transformer for Photorealistic Text-to-Image Synthesis

Stanford

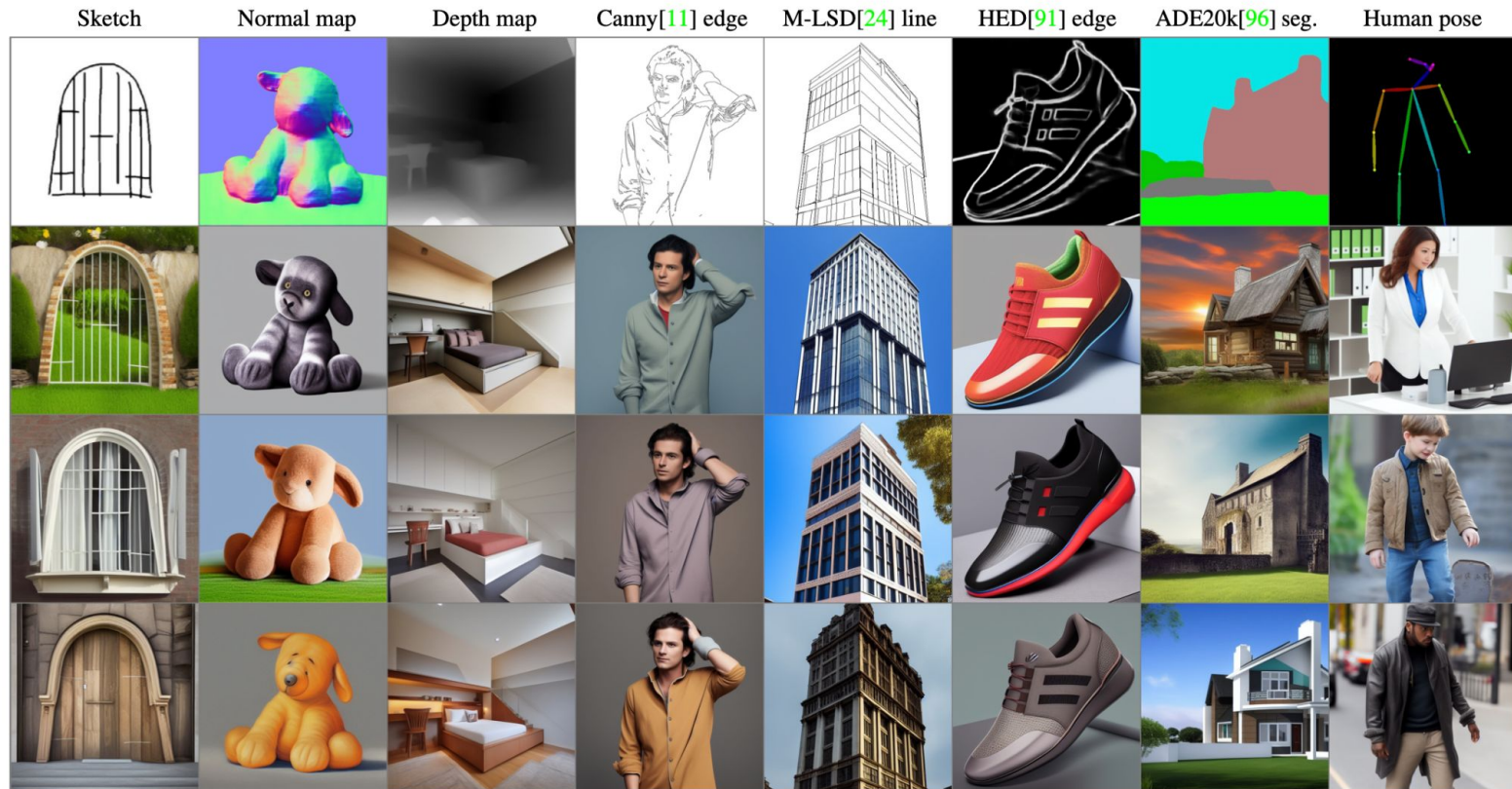
ControlNet for DiT



(a) PixArt- α

(c) ControlNet-Transformer

ControlNet for DiT



Transformer for Recognition: Whisper

Multitask training data (680k hours)

English transcription

🗣️ "Ask not what your country can do for ..."

📄 Ask not what your country can do for ...

Any-to-English speech translation

🗣️ "El rápido zorro marrón salta sobre ..."

📄 The quick brown fox jumps over ...

Non-English transcription

🗣️ "언덕 위에 올라 내려다보면 너무나 넓고 넓은 ..."

📄 언덕 위에 올라 내려다보면 너무나 넓고 넓은 ...

No speech

🔊 (background music playing)

📄 ∅

Sequence-to-sequence learning

